

FinalProject1

June 12, 2025

1 Final Project 1 (Tutorial using original image)

```
[1]: import numpy as np
import sep
```

```
[2]: import astropy.io.fits as fits
import matplotlib.pyplot as plt
from matplotlib import rcParams

%matplotlib inline

rcParams['figure.figsize'] = [10., 8.]
```

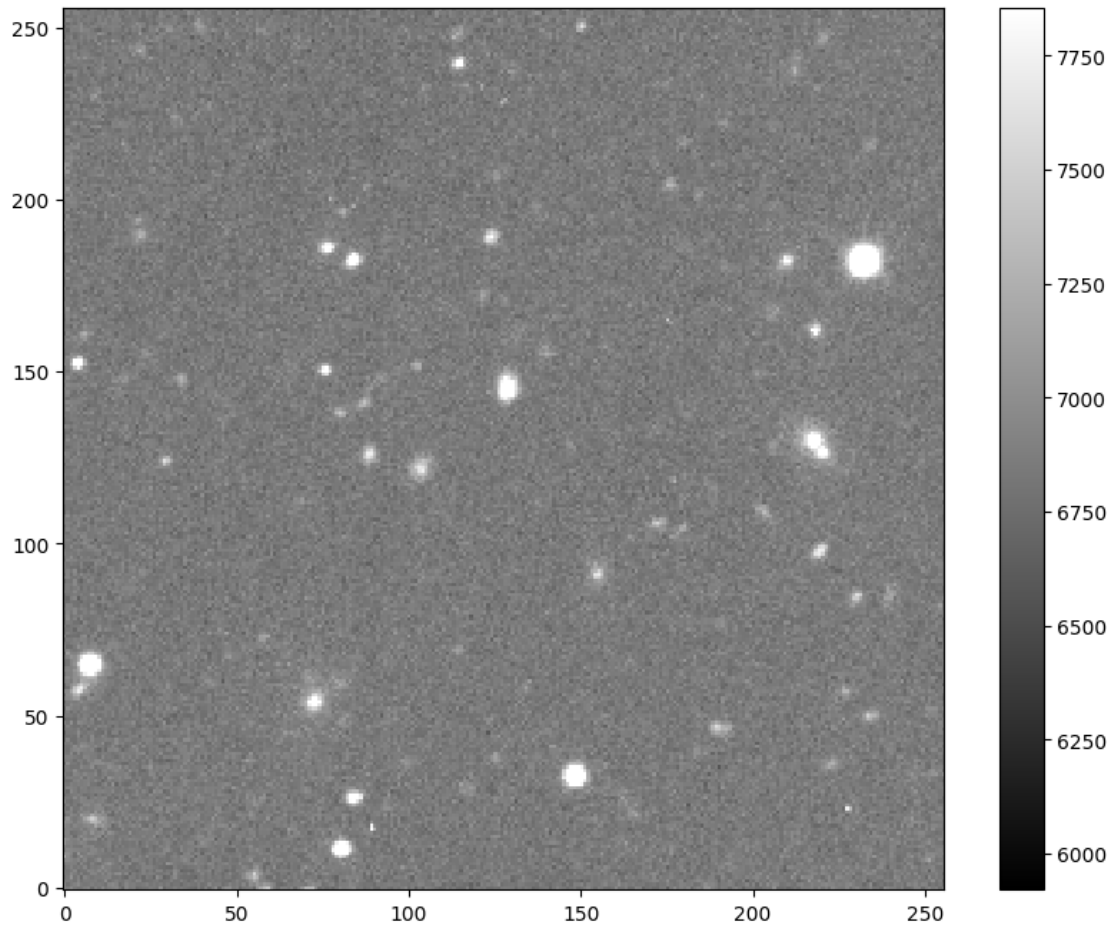
1.1 Read image into standard 2-d numpy array

```
[18]: fits_image_filename = 'image.fits'
with fits.open(fits_image_filename) as hdul: # open a FITS file
    data = hdul[0].data
```

1.2 Show the image and save as image_plot.png

```
[4]: m, s = np.mean(data), np.std(data)
plt.imshow(data, interpolation='nearest', cmap='gray', vmin=m-s, vmax=m+s,
           origin='lower')
plt.colorbar();

plt.savefig('image_plot.png')
```



1.3 Measure a spatially varying background on the image

```
[5]: bkg = sep.Background(data)
```

1.4 Get a “global” mean and noise of the image background:

```
[6]: print(bkg.globalback)
      print(bkg.globalrms)
```

```
6852.04931640625
65.46174621582031
```

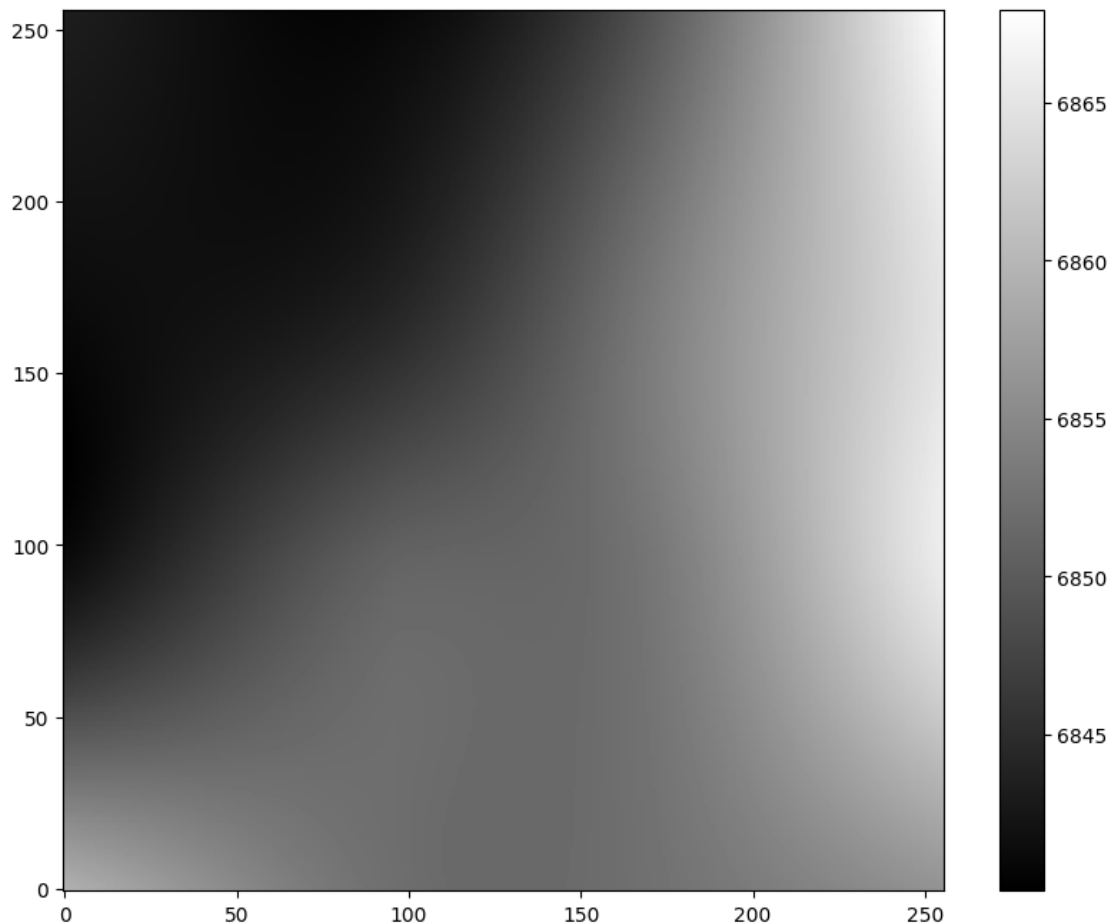
1.5 Evaluate background as 2-d array, same size as original image

```
[7]: bkg_image = bkg.back()
```

1.6 Show the background and save as background_plot.png

```
[8]: plt.imshow(bkg_image, interpolation='nearest', cmap='gray', origin='lower')
plt.colorbar();

plt.savefig('background_plot.png')
```



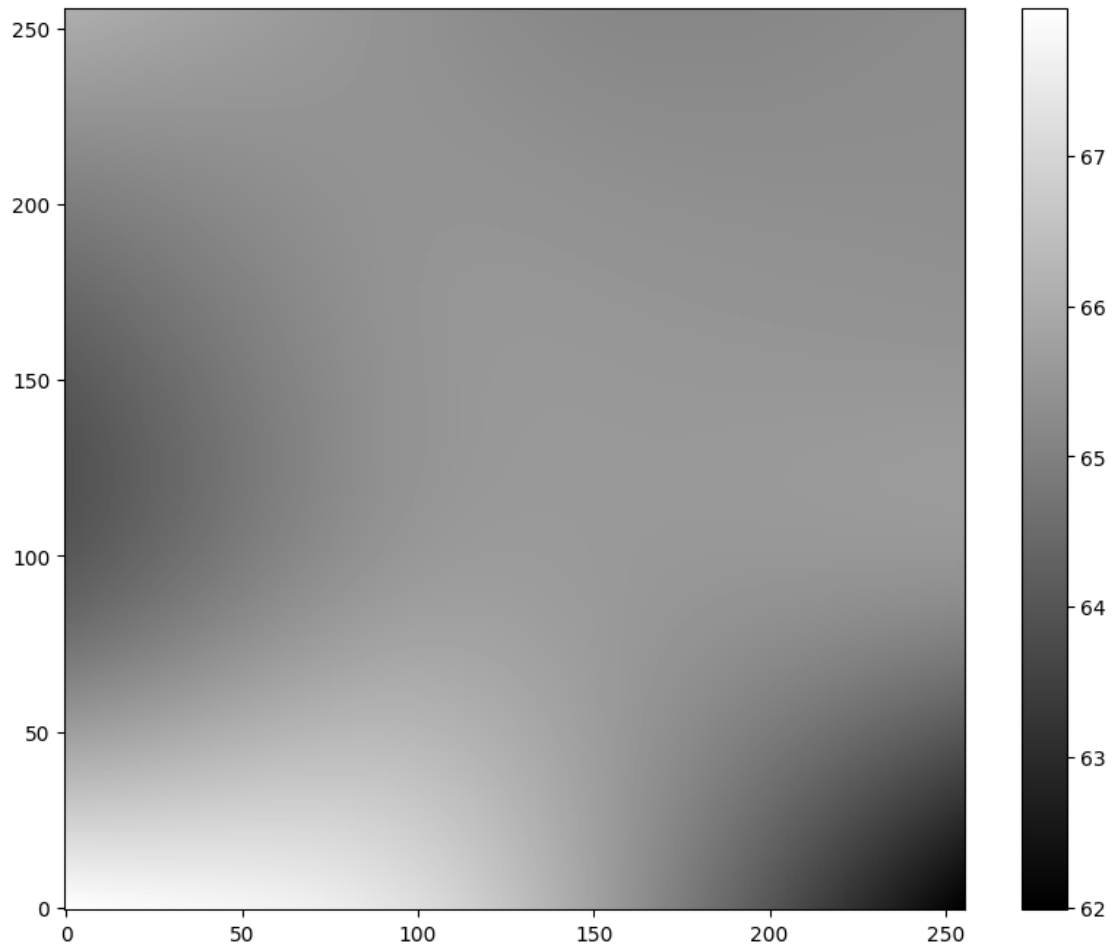
1.7 Evaluate the background noise as 2-d array, same size as original image

```
[9]: bkg_rms = bkg.rms()
```

1.8 Show the background noise and save the figure as background_noise.png

```
[10]: plt.imshow(bkg_rms, interpolation='nearest', cmap='gray', origin='lower')
plt.colorbar();

plt.savefig('background_noise.png')
```



1.9 Subtract the background

```
[11]: data_sub = data - bkg
```

1.10 Detect objects and set the detection threshold to be a constant value of 1.5 standard deviations

```
[12]: objects = sep.extract(data_sub, 1.5, err=bkg.globalrms)
```

1.11 Print how many objects were detected

```
[13]: len(objects)
```

```
[13]: 69
```

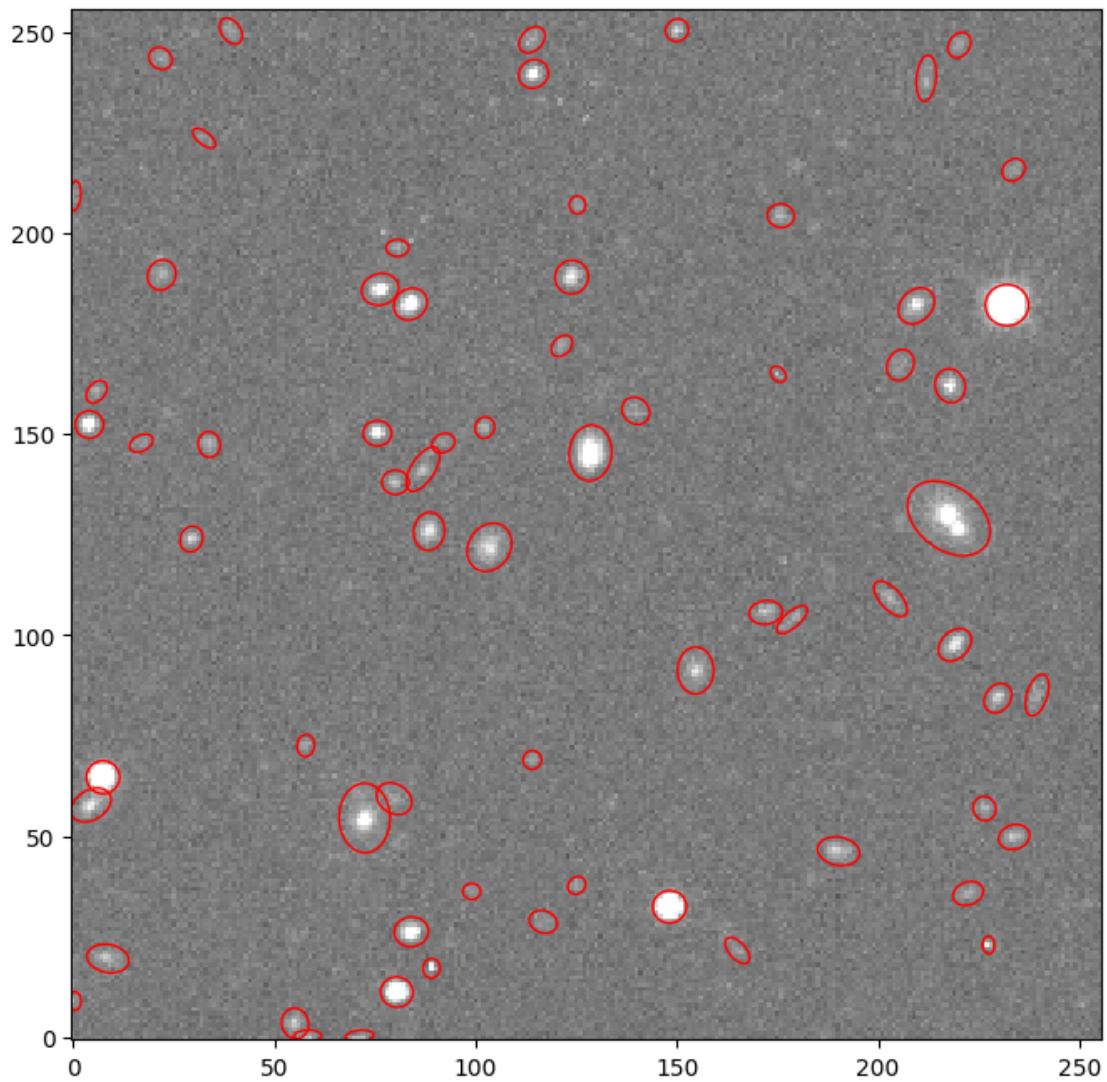
1.12 Over-plot the object coordinates with some basic shape parameters on the image and then save figure as image_coordinates.png

```
[14]: from matplotlib.patches import Ellipse

# plot background-subtracted image
fig, ax = plt.subplots()
m, s = np.mean(data_sub), np.std(data_sub)
im = ax.imshow(data_sub, interpolation='nearest', cmap='gray',
               vmin=m-s, vmax=m+s, origin='lower')

# plot an ellipse for each object
for i in range(len(objects)):
    e = Ellipse(xy=(objects['x'][i], objects['y'][i]),
                width=6*objects['a'][i],
                height=6*objects['b'][i],
                angle=objects['theta'][i] * 180. / np.pi)
    e.set_facecolor('none')
    e.set_edgecolor('red')
    ax.add_artist(e)

plt.savefig('image_coordinates.png')
```



1.13 List the available object fields

```
[15]: objects.dtype.names
```

```
[15]: ('thresh',  
      'npix',  
      'tnpix',  
      'xmin',  
      'xmax',  
      'ymin',  
      'ymax',  
      'x',  
      'y',
```

```

'x2',
'y2',
'xy',
'errx2',
'erry2',
'errxy',
'a',
'b',
'theta',
'cxx',
'cyy',
'cxy',
'cflux',
'flux',
'cpeak',
'peak',
'xcpeak',
'ycpeak',
'xpeak',
'ypeak',
'flag')

```

1.14 Perform simple circular aperture photometry with a 3 pixel radius at the locations of the objects

```
[16]: flux, fluxerr, flag = sep.sum_circle(data_sub, objects['x'], objects['y'], 3.0,
      ↪err=bkg.globalrms, gain=1.0)
```

1.15 Show the first 10 objects results:

```
[17]: for i in range(10):
      print("object {:d}: flux = {:.f} +/- {:.f}".format(i, flux[i], fluxerr[i]))
```

```

object 0: flux = 2249.159297 +/- 291.027802
object 1: flux = 3092.220430 +/- 291.592204
object 2: flux = 5949.868379 +/- 356.562003
object 3: flux = 1851.426582 +/- 295.028816
object 4: flux = 72736.386914 +/- 440.172206
object 5: flux = 3860.756152 +/- 352.163162
object 6: flux = 6418.913789 +/- 357.458973
object 7: flux = 2210.707656 +/- 350.791223
object 8: flux = 2741.607227 +/- 352.277746
object 9: flux = 20916.875566 +/- 376.966138

```